

Access modifiers in Java 7

* All access modifiers describe the **accessibility level of a class OR the member of the class (field + methods)**.

* In terms of accessibility, Java has provided 4 access modifiers :

1) private (within the same class only)
2) default (within the same package (buddy) only)
3) protected (either the same package as well as from another package but using inheritance)
4) public (from everywhere)

Access modifier	Within the same class	Within the same package	From Another Package	From everywhere (the internet)
private	YES	NO	NO	NO
default	YES	YES	NO	NO
protected	YES	YES	YES (using inheritance)	NO
public	YES	YES	YES	YES

private access modifier :

It is the most restrictive access modifier because the member declared as private can't be accessible from outside of the class.

As Java can't define an outer class as a private or protected or static. Generally we should declare the non static fields with private access modifier (Data hiding).

In Java outer classes can be declared as public, default, final, sealed and non-sealed modifier only.

default access modifier :

```
class Test {
    //data Test
    //...
    //...
    //...
}
//default access modifier
//...
//...
//...
}
```

It is an access modifier which is less restrictive than private. It is both kind of access modifier where the member declared as default is accessible from within the class and also from outside the class before the class name. field name or method name then by default it would be default.

As far as its accessibility is concerned, default members are accessible within the same sub-package only. It is also known as package package modifier.

```
package com.kit;
public class Test {
    //...
    //...
    //...
}
//package com.kit;
//...
//...
//...
}
```

public class Test {
 //...
 //...
 //...
}

Note : When Java is available in the same package so, can access default member.

```
package com.kit;
//...
//...
//...
}
```

It is an access modifier which is less restrictive than default because the member declared as default is accessible from the outside of the package (buddy) but by using inheritance.

```
package com.kit;
public class Test {
    //...
    //...
    //...
}
//package com.kit;
//...
//...
//...
}
```

Note : If a class is available in another package so it can access protected member by using inheritance only.

```
package com.kit;
//...
//...
//...
}
```

public :

It is an access modifier which does not contain any kind of restriction that is the member declared as public can be accessible from everywhere without any restriction.

According to Object Oriented java we should declare the classes and methods as public where as fields must be declared as private or protected according to the requirement.

Note : If a method is used for internal purpose only (like validation) then we can declare that method as private method or a class method.

Access modifiers : private, default, protected and public

Remembering all are non access modifiers OR simply modifiers like final, static and so on.

When to give higher properties than static fields only by using constructor instead of static class.

If we want to give our object properties (instance variables OR non static fields) then we should generate the following method in our class then right class.

Now with the help of following method we need not to write the display kind of method to print the object properties in static variable.

As soon to generate the following method we need to follow the steps.

Right click on the program -> source -> generate toString()

As soon to add this following method, we need to give the corresponding object reference by using System.out.println() statement.

Manager in a new Manager() ;

System.out.println(); (Calling toString() method of Manager class)

toString() = new toString();

System.out.println(); (Calling toString() method of Employee class)

```
package com.kit;
public class Manager {
    //...
    //...
    //...
}
//package com.kit;
//...
//...
//...
}
```

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}

package com.kit;
public class Manager {
 //...
 //...
 //...
}
//package com.kit;
//...
//...
//...
}